"وظيفته ببساطة إنه بيعمل "كوبرا Node.js. لمكتبة (Object-Relational Mapper) **ORM** هو عبارة عن **Sequelize** (SQL Tables). وبين لغة قواعد البيانات (Objects) بين لغة الجافا سكريبت (Bridge).

بدل ما تكتب `SELECT * FROM products` .. إنت بتنادي ميثود اسمها `Product.findAll()` .

---

1. إزاي بيحول حياتك؟ (The Magic)

من غير Sequelize (اللي إنت بتعمله دلوقتي):

JavaScript

```
db.execute("INSERT INTO products (title, price) VALUES (?, ?)", [title,
price]);
```

مع **Sequelize**:

JavaScript

```
Product.create({
  title: 'A Book',
  price: 12.99
});
```

---

2. المميزات اللي تخليك "تدلعه":

- **وداعاً للـ SQL Queries المعقدة:** مش محتاج تشغل بالك بفتح وقفل الـ Connections ولا الـ Syntax الرخم بتاع الـ Joins. هو بيعمل ده بدالك.
- **الـ Migrations:** دي أهم ميزة. لو عايز تغير في جدول (تضيف عمود مثلاً)، Sequelize بيعملك ملف "تأريخ" للتغييرات دي عشان لو نقلت مشروعك لسيرفر تاني، الجداول تتبني لوحدها بنفس المواصفات.
- **الـ Relationships (العلاقات):** لو عايز تربط "المستخدم" بـ "المنتجات" بتاعته، بتكتب سطر واحد: `User.hasMany(Product)` . هو بقى هيروح يعمل الـ Foreign Keys والـ Joins لوحده في الخلفية.
- **دعم قواعد بيانات كتير:** لو قررت النهاردة تسيب MySQL وتروح لـ PostgreSQL أو SQLite، مش هتغير كود الأبلكيشن! بس هتغير سطر واحد في الإعدادات، والـ ORM هيترجم لغة الـ SQL الجديدة لوحده.

---

3. العيوب (عشان نكون صرحاء):

- **الأداء (Performance):** في الـ Queries المعقدة جداً، الـ Raw SQL (اللي بتكتبه بإيدك) بيبقى أسرع "سنة" لأن الـ ORM بياخد وقت بسيط عشان يترجم الكود لـ SQL.
- **منحنى التعلم:** هتحتاج تتعلم الـ Functions بتاعته والـ Configuration في الأول، بس بعد كدة هترتاح جداً.

```
const Sequelize = require("sequelize");



const sequelize = new Sequelize(

  process.env.DB_NAME,

  process.env.DB_USER,

  process.env.DB_PASSWORD,

  { host: process.env.DB_HOST, dialect: "mysql" },

);



module.exports = sequelize;
```

---

2. تعريف الموديل (Defining the Model)

- `sequelize.define("product", { ... })`:
  - كلمة `product`: ده اسم الموديل. Sequelize بياخد الاسم ده ويحوله "جمع" أوتوماتيك ويسمي الجدول في الداتابيز `products`.
  - **الأوبجكت** `{ ... }`: ده اللي بنرسم فيه شكل الأعمدة (Columns).

---

3. تفصيص الأعمدة (Column Definitions)

كل عمود جواه "عقد" مواصفات:

- `id`:
  - `type: Sequelize.INTEGER`: بقوله ده رقم صحيح.

- `autoIncrement: true` : يعني "عدّاد"؛ كل منتج جديد هياخد رقم لوحده (1, 2, 3...) مش محتاج أنا أديله رقم.
- `primaryKey: true` : يعني ده "البصمة" أو الرقم الفريد اللي مبيتكررش، وعن طريقه بنوصل للمنتج ده بسرعة.
- `allowNull: false` : ممنوع يكون فاضي.

- `title` :
  - مكتوبة باختصار `Sequelize.STRING` . ده معناه إنه هياخد المواصفات الافتراضية (Default) وهي إنه نص (VARCHAR) وممكن يكون فاضي.
- `price` :
  - `type: Sequelize.DOUBLE` : الـ Double عشان السعر ممكن يكون فيه كسر (مثلاً 9.99). بيستحمل الأرقام العشرية.
  - `allowNull: false` : لأن مفيش منتج من غير سعر.
- `imageUrl` & `description` :
  - الاتنين `STRING` (نصوص).
  - `allowNull: false` : عشان نضمن إن الداتابيز ديما شايلة بيانات كاملة للمنتج.

---

4. التصدير (The Export)

- `module.exports = product` : عشان نعمل **Controllers** بنخرج الموديل ده عشان نقدر نستخدمه في الـ Create أو Find) أو في الـ **app.js** (عشان نعمل الـ Sync).

ملخص المعمارية في السطرين دول:

الكود ده وظيفته **"Mapping"**؛ يعني بيعمل خريطة. هو مش كود بينفذ حاجة دلوقتي، هو "تعريف" بيقول لـ Sequelize:

> "يا Sequelize، لما تروح الداتابيز وتعمل `sync` .. اعملي جدول بالمواصفات دي بالظبط، ولما أقولك `Product.create` خزن البيانات في الأعمدة دي."

---

1. `sequelize.sync()` (المُنسق)

دي أهم ميثود في Sequelize. وظيفتها إنها تعمل "جرد" (Mapping):

- بتروح تبص على كل الـ **Models** اللي إنت عرفتها في الكود (زي `Product`).
- بتروح للداتابيز (MySQL) وتشوف: "هل الجداول دي موجودة بنفس المواصفات؟".
- لو مش موجودة، Sequelize بيكريتها أوتوماتيك بأمر `CREATE TABLE IF NOT EXISTS` .

2. الـ `then((result) => { ... }).` (شهادة النجاح)

الـ `sync()` دي ميثود **Asynchronous** (بتاخد وقت)، فعشان كدة بترجع **Promise**.

- الكود اللي جوه الـ `.then` مش هيتنفذ غير لما Sequelize يخلص شغله مع الداتابيز ويرد بـ "تمام يا ريس، الجداول جاهزة".
- إحنا حطينا السطر ده هنا "مخصوص" عشان نضمن إن السيرفر مش هيقوم ويستقبل طلبات غير : `app.listen(3000)` أول يوزر هيدخل الموقع هيجيله .. `sync` والسيرفر قام والداتابيز لسه مخلصتش `.then` والداتابيز شغالة. لو حطيته بره الـ Error فوراً.

3. الـ `console.log(result)` (تقرير المعمل)

الـ `result` ده أوبجكت ضخم فيه تفاصيل كل اللي Sequelize عمله (أسامي الجداول اللي اتكريتت، الكونفجريشن، إلخ).

> **نصيحة صنايعي:** في الشغل الحقيقي بنمسح السطر ده أو بنعمله `comment` عشان بيملا الـ Terminal كلام كتير ملوش لازمة، كفاية تطبع رسالة "Database Connected!".

4. الـ `.catch((err) => ...)` (الـ Black Box)

لو الداتابيز فيها مشكلة (زي الـ `Access Denied` اللي طلعلك من شوية، أو الباسورد غلط، أو الداتابيز مش موجودة أصلاً):

- الـ `sync` هيفشل.
- الكود هينط على الـ `.catch`.
- السيرفر **مش هيفتح** (`app.listen` مش هتتنفذ)، وده الصح عشان متبقاش فاتح موقع "خربان" من جوه.

الفرق بين الرد في SQL والـ Sequelize

دي حتة "فنية" لازم تاخد بالك منها:

- **في الـ SQL القديم (`db.execute`):** كان بيرجع لك Array جواها Array تانية (الـ Rows)، فكنت بتعمل `rows[0]`.
- **في Sequelize:** الـ `findAll` بترجع لك **Array of Objects** جاهزة للاستخدام فوراً في الـ View بتاعك.

تريك الـ "JSON" (لو عايز بيانات صافية):

الـ `products` اللي بترجع من `findAll` بتبقى Objects "ذكية" تابعة لـ Sequelize (فيها ميثودز تانية). لو عايزها تبقى بيانات "خام" (Plain JavaScript Object) عشان تتعامل معاها بسهولة، ممكن تستخدم:

JavaScript

```
Product.findAll({ raw: true })
```

> **"الـ `findAll` هي 'الشبكة' اللي بترميها في بحر الداتابيز.. لو رميتها فاضية تجيب كل السمك، ولو حطيت فيها طعم (`where`) تجيب اللي إنت عايزه بس!"**

- **الـ SQL القديم (`db.execute`):** كان بيرجع لك `[[rows], fieldData]` . فكنت بتضطر تاخد `rows[0]` عشان توصل للمنتج.
- **الـ `findByPk` :** هي ذكية كفاية إنها تطلع لك المنتج بره الـ Array وتديهولك جاهز. يعني بتتعامل مع `product.title` مباشرة مش `product[0].title` .

المرحلة الأولى: الـ Retrieval (الاستدعاء)

JavaScript

```
Product.findByPk(prodId)
```

الـ Sequelize هنا بيعمل `SELECT * FROM products WHERE id = ?` .

- **ليه بنعمل كدة؟** عشان نتأكد إن المنتج لسه موجود قبل ما نعدل، وعشان الـ Sequelize يجهز لنا "Object" ذكي مربوط بالداتابيز.

---

3. المرحلة الثانية: الـ Property Assignment (التعديل في الميموري)

JavaScript

```
.then((product) => {
  product.title = updatedTitle;
  // ...
  return product.save();
})
```

الحتة دي هي "السر":

- لما بتقول `product.title = ...` إنت لسه ملمستش الداتابيز. إنت بتغير في الـ **JS Object** اللي في الميموري بتاعة السيرفر.
- الـ `product.save()` : دي ميثود "عبقرية". Sequelize بيعمل حاجة اسمها **Dirty Checking**. بيقارن بين البيانات القديمة اللي جابها من الداتابيز وبين التغييرات اللي إنت عملتها.
- لو لقى إن `title` بس هو اللي اتغير، هيبعت أمر `UPDATE` للـ `title` بس. لو مفيش حاجة اتغيرت خالص، ممكن ميبعتش حاجة أصلاً عشان يوفر وقت!

---

4. المرحلة الثالثة: الـ Promise Chaining (التسلسل)

JavaScript

```
.then((result) => res.redirect("/admin/products"))
```

- **ليه عملنا `return product.save()`** ؟ عشان `save` دي بياخد وقت (Async). لما بنعمل `return` إنت بتقول للـ Promise: "استني لما السيف يخلص، وبعدين ادخلي في الـ `then.` اللي بعدها".
- الـ `result` اللي راجع ده هو المنتج بعد التعديل، بس إحنا مش محتاجينه في حاجة غير إننا نتأكد إن العملية تمت، فنقوم عاملين `redirect` فوراً عشان اليوزر يشوف النتيجة.

---

5. ليه الطريقة دي أحسن من الـ Raw SQL؟

في الـ SQL العادي كنت هتكتب سطر طويل عريض: `UPDATE products SET title=?, price=? ... WHERE id=?` **العيوب:** لو نسيت عمود واحد الدنيا تبوظ، ولو الداتا هي هي برضه هيعمل Update ويستهلك موارد.

في Sequelize:

1. **أمان (Security):** محمي تماماً من الـ SQL Injection.
2. **سهولة (Maintainability):** لو زودت عمود في الموديل بكرة، التعديل هنا هيبقى سطر جافا سكريبت بسيط.
3. **الـ Timestamps:** الـ Sequelize هيحدث حقل الـ `updatedAt` لوحده بالثانية والدقيقة اللي حصل فيها التعديل.

الـ `destroy()` (الضربة القاضية)

JavaScript

```
return product.destroy();
```

بمجرد ما الـ Object بقى في إيدك، بتنادي عليه ميثود الـ `destroy()` .

- Sequelize بيروح يحول الأمر ده لـ SQL Query: `DELETE FROM products WHERE id = ?` .
- الـ `return` هنا مهمة جداً لأن `destroy` بترجع **Promise**. إنت كدة بتعمل "تسلسل" (Chaining) عشان تروح للـ `then.` اللي بعدها بس لما المسح يخلص فعلياً.

---

1. القاعدة الذهبية: مين اللي بيشيل الـ "مفتاح"؟ (Foreign Key)

في Sequelize، الجداول نوعين: **Source** (المصدر) و **Target** (الهدف).

- لما تقول `A.belongsTo(B)` .. الـ **Foreign Key** بيروح يتحط في جدول **A**.
- دي أهم معلومة: الـ `belongsTo` هي اللي ديماً "بتخرم" الجدول بتاعها وتحط فيه الـ ID بتاع الجدول التاني.

---

2. أنواع العلاقات (الأربعة الكبار)

أ- One-to-One (واحد لواحد)

زي المستخدم وبروفايله.

- `User.hasOne(Profile)`
- `Profile.belongsTo(User)`
  - **النتيجة:** جدول الـ `Profiles` هيبقى فيه عمود اسمه `userId` .

ب- One-to-Many (واحد لكتير)

دي أشهر علاقة (زي اليوزر والمنتجات بتاعته).

- `User.hasMany(Product)`
- `Product.belongsTo(User)`
  - **النتيجة:** كل منتج في جدول الـ `Products` هيبقى معاه `userId` عشان نعرف مين "صاحبه".

ج- Many-to-Many (كتير لكتير)

زي الطلاب والكورسات (الطالب بياخد كذا كورس، والكورس فيه كذا طالب).

- `Student.belongsToMany(Course, { through: 'Enrollments' })`
  - **الزتونة:** هنا لازم "جدول وسيط" (Through Table). Sequelize هيكريهولك أوتوماتيك ويسميه `Enrollments` .

---

3. الميثودز السحرية (The Magic Methods)

دي الحتة اللي بتخليك تحس إنك ساحر. بمجرد ما تعمل `User.hasMany(Product)` ، الـ Sequelize بيضيف "ميثودز" من تحت الأرض للأوبجكت بتاع الـ User:

- `user.getProducts()` : هات لي كل منتجات اليوزر ده.
- `user.createProduct({ title: '...' })` : ضيف منتج جديد واربطه باليوزر ده أوتوماتيك (من غير ما تبعت بإيدك الـ `userId`!).
- `user.addProducts([p1, p2])` : اربط منتجات موجودة أصلاً باليوزر ده.

---

4. الـ Eager Loading (هات كله في خبطة واحدة)

بدل ما تروح تجيب اليوزر، وبعدين تعمل Query تانية تجيب منتجاته.. إنت بتعمل **Include**:

```
User.findAll({
  include: [Product] // هات لي اليوزر ومعاه لستة منتجاته في Query واحدة!
})
```

ده بيترجم في الـ SQL لـ `LEFT OUTER JOIN` .

---

5. الـ Aliases (الأسماء الدلع)

لو عايز تسمي العلاقة اسم غير اسم الجدول (مثلاً اليوزر عنده "طلبات" مش "أوردرات"):

```
User.hasMany(Order, { as: 'Orders' });
```

ساعتها الميثود السحرية هيبقى اسمها `user.getOrders()` .

---

1. `Product.belongsTo(User, ...)` (المنتج يتبع مستخدم)

السطر ده هو "المعلم" اللي بيغير شكل جدول الـ `products` :

- **الـ Foreign Key:** أوتوماتيك، Sequelize هيروح يضيف عمود جديد في جدول الـ `products` اسمه `userId` .
- `constrains: true` : مش موجود أصلاً في جدول الـ `userId` دي معناها "اربطهم بجد"؛ يعني مينفعش تضيف منتج بـ `Users` .
- `onDelete: "CASCADE"` : دي أهم حتة "أمان". معناها لو اليوزر ده اتمسح من الموقع، **امسح كل منتجاته معاه** أوتوماتيك. متبقاش فيه منتجات "يتيمة" في الداتابيز ملهاش صاحب.

---

2. `User.hasMany(Product)` (المستخدم عنده منتجات كتير)

السطر ده مش بيضيف أعمدة في الداتابيز، لكنه بيضيف "قوة خارقة" للموديل بتاعك في الكود (Magic Methods):

- بيخليك تقدر تقول: `user.getProducts()` عشان تجيب لستة حاجاته.
- بيخليك تقدر تقول: `user.createProduct({ title: '...' })` وهو لوحده هيحط الـ `userId` الصح للمنتج الجديد.

3. ليه بنكتب السطرين مع بعض؟ (The Circular Link)

لو كتبت `belongsTo` بس، العلاقة هتبقى من طرف واحد (تقدر تعرف صاحب المنتج، بس متقدرش تعرف اليوزر عنده إيه بسهولة). لما بتكتب الـ **Pair** (الاثنين مع بعض)، إنت بتعمل "ربط مزدوج":

1. **في الداتابيز:** الـ Foreign Key بيبقى مكانه في الـ `Products` .
2. **في الكود:** بتقدر تمشي في الطريقين (من اليوزر للمنتج، ومن المنتج لليوزر).

---

1. إزاي Sequelize بيتعامل مع الـ `req.user` ؟

لما بتعمل `User.findByPk(1)` في الـ Middleware، إنت مش بترجع مجرد "بيانات" (JSON)، إنت بترجع **Sequelize Instance**.

- **الفرق الجوهري:** لو البيانات راجعة من SQL عادي، هتبقى عبارة عن Object فيه `name` و `email` بس. لكن في Sequelize، الـ Object ده بيبقى متوصل بـ "الـ Prototype" بتاع الموديل، وده اللي بيخليه يقدر ينادي على ميثودز زي `.save()` أو `.destroy()` .
- **الحقن (Injection):** لما بتقول `req.user = user;` .. إنت كأنك ركبت "موتور" في الـ `req` . أي Controller هييجي بعد الميدل وير ده هيقدر ينده على `req.user` ويستخدم كل قوته.

---

2. إزاي الـ Associations بتضيف "الميثودز السحرية"؟

بمجرد ما كتبت السطرين دول:

JavaScript

```
Product.belongsTo(User);
User.hasMany(Product);
```

سيكويليز بيعمل حاجة اسمها **Mixins**. هو بيشوف اسم الموديل (`Product`) وبناءً عليه بيفصل لك ميثودز مخصوصة ويركبها في الـ `User Instance` .

الميثودز اللي بتنزل لليوزر أوتوماتيك:

بما إن الـ User عنده `hasMany` منتجات، سيكويليز بيضيف له:

1. **`.getProducts()`** : بدل ما تعمل `Product.findAll({where: ...})` .
2. **`.createProduct()`** : ودي الأهم! بتكريت منتج وبتربطه باليوزر ده في خطوة واحدة.
3. **`.addProducts()`** : لو عندك منتجات موجودة وعايز تنسبها لليوزر ده.